Sabine Mejia, Krish Chaudhary
CIS 5410
Professor Insup Lee

Pacemaker Project Final Report
Team: PacingAhead

I. System Specifications

In this project, we modeled, implemented, and validated a VVI mode pacemaker. The pacemaker paced and sensed from a random heart, which we designed to simulate the physiology of a real heart. In order to analyze the behavior of our heart and pacemaker and check that safety properties were upheld, we implemented an observer/monitor.

II. Formal Modeling

We began by creating formal models of the random heart, pacemaker, and observer. We used the UPPAAL tool to model timed automata for each component.

Global variables:

```
const int LRI = 1500; // Lower Rate Limit (40 bpm) - as specified by project instructions
const int URI = 333;  // Upper Rate Limit (180 bpm) - as specified by project instructions
const int HRI = 1600; // Hysteresis Rate Interval (~33 bpm)  - chosen to be slightly slower than LRI
const int VRP = 150;  // Ventricular Refractory Period (ms) - as specified by project instructions
```

They are used in the Ventricle controller's logic and allow us to verify the properties discussed in Section III.

Communication: RandomHeart, VentricleController, and Observer communicate over the broadcast channels VSense and VPace.

RandomHeart: Models a heart that can create its own ventricular event or register a paced ventricular event.

- Generates heartbeats between the interval minwait = 200 ms and maxwait = 2,000 ms
- We chose a high maxwait to model the condition of Bradycardia.
- Ready State: Able to receive a Pace from the VentricleController or emit a Sense to the VentricleController
- Invariant: t must be less than maxwait. If it hasn't received a Pace! It must transition out of maxwait by beating (emitting VSense!).
- Idle State: unable to receive a pace or emit a sense if the ventricular refractory period has not elapsed.
- Invariant: t must be greater than VRP. The heart must be incapable of receiving paces or sending senses for the duration of the VRP (150ms)

- VPace? Transition: the heart has received a pace from the VentricleController.
- VSense! Transition: the heart has beat on its own and is communicating that to the Observer and VentricleController.

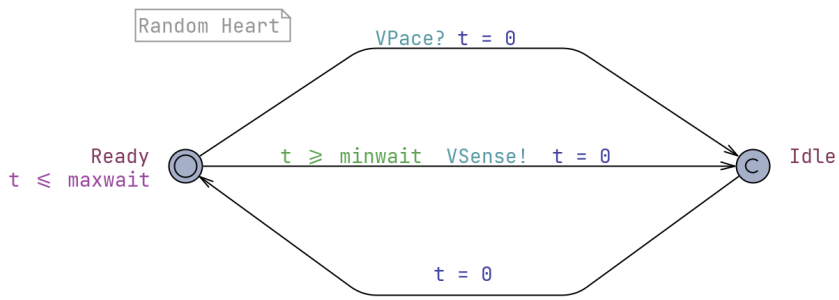

Figure 1: Timed Automata of UPPAAL Random Heart. The random heart produces beats at random intervals between minwait and maxwait.

VentricleController: Sends paces to the heart to produce a ventricular event. If hysteresis pacing is off, the VentricleController will ensure that a pace will be delivered before LRI if no pace has been sensed. VentricleController also has a hysteresis mode that is enabled each time a natural beat is sensed. In hysteresis mode, the VentricleController uses a Rate Interval equal to HRI, which is slightly slower than LRI.

- WaitRI State: VentricleController is waiting for a heartbeat or waiting to send one.
    - Invariant: the clock variable x must be less than or equal to RI. This ensures that if no beat is sensed, the VentricleController will pace.
- WaitVRP State: The VentricleController is waiting for VRP to elapse and will not register a sense or send a pace in this state.
    - Invariant: the clock variable x must be less than or equal to VRP.
- VPace! Transition: VentricleController has generated a ventricular event. Hysteresis pacing is enabled to false and VentricleController defaults to LRI.
- VSense? Transition: A natural beat has been sensed, HP is enabled.

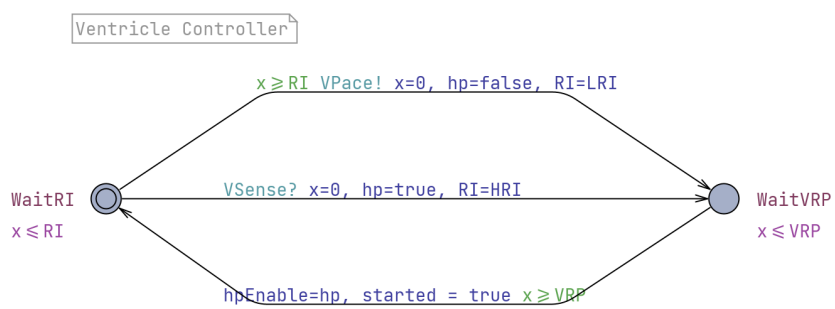

Figure 2: Timed Automata Model of Ventricle Controller in UPPAAL.

Observer: Records the duration (ms) between the previous and most recent ventricular event (either sensed or paced). Used to verify timing properties of the model.

- Resets tv = 0 on every ventricular event (sense or pace)
- moves into location VPaced exactly when a VPace? Depending on whether the RandomHeart or VentricleController has initiated a ventricular event.
- WaitRI State: tv is running in this state
- VSensed State: a committed location (no time can elapse in this state), immediately transitions out and restarts the tv clock
- VPaced State: also a committed location, immediately transitions out and restarts tv clock

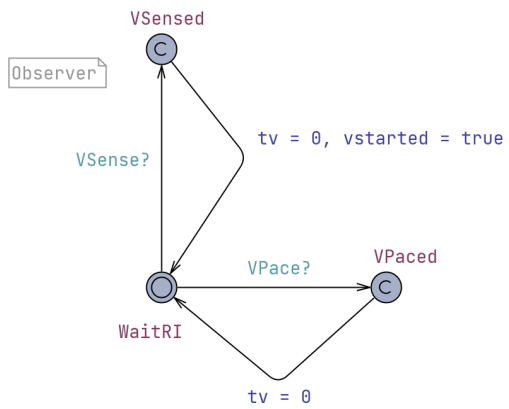

Figure 3: Timed Automata of Observer in UPPAAL.
The observer measures the time (ms) between paced or sensed ventricular events.

III. Formal Model Verification

Our model of the pacemaker system is formally verified by the model checking technique using the real-time model checker in UPPAAL. The following verification queries ensure that our Pacemaker is guaranteed to deliver paces at safe intervals.

We wrote the following 4 queries to ensure safety-critical timing properties:

1. **PropNoDeadlock**

Query: A[] not deadlock

- Meaning: In all reachable states of the system, the system never reaches a state of deadlock.
- Explanation: We want to ensure that deadlock is not possible on any path. If deadlocks exist, it is possible for the system to reach a state where no automaton can move, no time is allowed to pass, and no synchronizations are possible. In a pacemaker model, this would correspond to the heart ceasing to beat, the pacemaker not sensing or pacing, and the model becoming frozen.

2. **PropLRI**

Query: A[] (!VentricleController.hpEnable imply Observer.tv <= LRI)

- Meaning: In normal pacing mode (hysteresis pacing not enabled), the pacemaker should never allow the ventricular-ventricular interval to exceed LRI.

- Explanation: If hysteresis is not enabled, the pacemaker must guarantee a pace before LRI is exceeded.

3. **PropVRP**

Query: A[] ((VentricleController.WaitRI && VentricleController.started) imply VentricleController.x >=VRP)

- Meaning: The pacemaker should not sense beats before VRP has elapsed.
- Explanation: After a ventricular event, there is a refractory period where the heart repolarizes. We do not want the pacemaker picking up on ECG noise from the heart and misclassifying ventricle activity during this period as a beat.

4. **PropHRI**

Query: A[] (VentricleController.hpEnable imply Observer.tv <=HRI)

- Meaning: If hysteresis pacing is activated, a ventricular pace or sense should be no later than HRI.
- Explanation: If a natural ventricular event is sensed, hysteresis pacing tells the VentricleController to wait slightly longer than the LRL before sending a Pace! If the VentricleController senses an event, it will enable hysteresis pacing and set the rate interval to HRI (Hysteresis Rate Interval) instead of LRI. This encourages the heart to beat naturally.

Results of Formal Model Verification:

UPPAAL's real time model checker tests whether a system design behaves correctly under all possible executions. Figure 4 below demonstrates that for our Pacemaker model, UPPAAL's real-time model checker found no traces that violate our 4 verification queries.

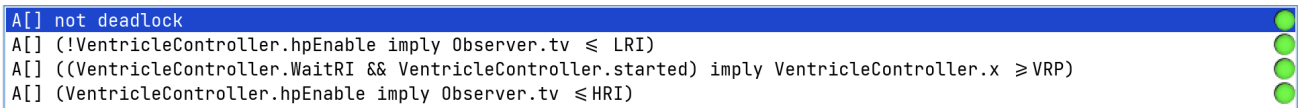

Figure 4: Verified UPPAAL queries. Each query checks that a safety-critical timing property is guaranteed.

IV. Code Generation

We followed the Thread-Per-EFSM code generation technique described in the code generation lecture slides. Each component of our system (randomHeart, ventricleController, and Monitor) runs its own program concurrently as its own EFSM. These components communicate through event channels (MQTT topics and digital I/O), as described in the Thread-per-EFSM approach. Our code generation takes the distributed coordination approach from the lecture, as no centralized coordinator manages transition selection, and each EFSM executes independently.

The synchronization edges in UPPAAL (VSense!, VPace?) are implemented using digital I/O pulses and MQTT messages, which act as distributed event queues. Guard conditions on clocks and variables in the timed automata translate directly into if conditions on hardware timers (millis()), and update actions correspond to state updates in the code. Because each

EFSM's structure, guards, and transitions are preserved in the generated code, our implementation is a correct translation of the UPPAAL timed automata model.

V. Code Structure

RandomHeart (Arduino)

The RandomHeart communicates exclusively with the VentricleController through digital I/O signaling. The RandomHeart chooses a random interval and produces a beat if the interval has elapsed without an interrupting pace from the VentricleController. If the heart receives a pace, it will reset its clock and start a new random interval. The RandomHeart runs an infinite loop, which implements the states, guards, and transitions present in the UPPALL Model:

- Time_ms = millis() → UPPAAL clock t
- transition VPace? → digitalRead(pacerPin)
- transition VSense! → digitalWrite(ledPin, HIGH)
- guard t ≥ minwait → (rand_interval <= time_ms - lastBeat)

VentricleController (Arduino)

The Ventricle Controller uses digital I/O signaling to communicate with the random heart and MQTT protocol to communicate with the Codio dashboard. It implements two FreeRTOS tasks: one polls for MQTT, and the other runs the controller logic.

FreeRTOS tasks:

- TaskMQTT (keeps MQTT connection with the Observer alive)
- TaskController (corresponds to the VentricleController's EFSM thread)

In TaskController(), the VentricleController runs an infinite loop which implements the states, guards, and transitions present in the UPPALL Model:

- if (digitalRead(sensePin) == HIGH) → UPPAAL VSense?
- if (time_ms - lastBeat >= RI) → guard x ≥ RI
- vTaskDelay(pdMS_TO_TICKS(VRP)) → invariant x ≤ VRP
- actions → hpEnable=true/false
- mqtt_client.beginMessage(); → UPPAAL synchronization channel with observer

Observer (Heart Monitor)

The observer receives updates regarding ventricular activity from the ventricle controller via MQTT. The observer uses threading to allow the MQTT client to continuously receive messages while simultaneously analyzing the pacemaker's activity. Every P seconds, the monitor analyzes the pacemaker system's behavior within a window of the past W seconds. The monitor records the number of paces, senses, pace ratio, average heart rate. If timing properties (PropLRI, PropHRI, PropVRP) are violated, the heart's pace ratio is too high, or the heart is beating too quickly, it prints corresponding alert messages to the terminal window.

Threads:

- mqtt_handler (keeps the MQTT connection with the Observer alive)
- analysis_thread (corresponds to the Observer's EFSM thread)

In the MessageRelayCallback::message_arrived() callback and the window_analysis() loop, the Observer EFSM runs continuously and implements the states, guards, and transitions present in the UPPAAL model.

- if (type == "VSense") → UPPAAL VSense? transition
- last_event = now_ms → corresponds to UPPAAL update tv = 0
- if (interval <= VRP - tolerance) → UPPAAL guard violation of x ≥ VRP
- if (interval > HRI + tolerance) → UPPAAL deadlock guard violation

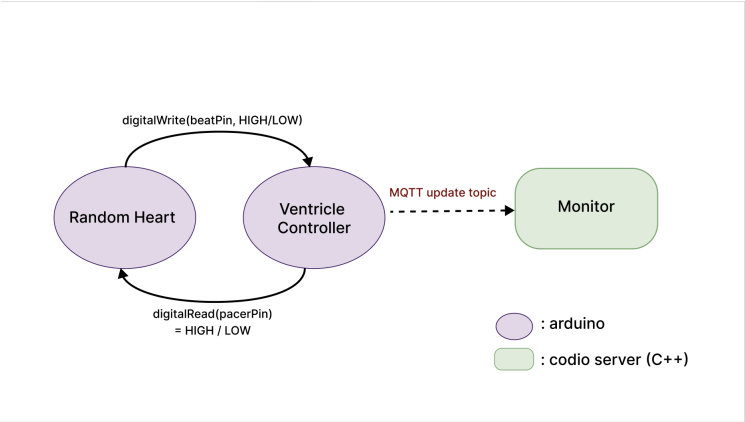

Figure 5: Structure of implemented code

VI. Testing

A. Random Testing

Our system inherently enables Random Testing because the RandomHeart generates ventricular intervals uniformly at random between 200 ms and 3000 ms. This nondeterministic behavior produces unpredictable sequences of sensed beats and paced beats. In order to increase coverage, we ran our simulation for ~6 minutes and monitored it for hundreds of ventricular events, allowing the Observer to evaluate repeated occurrences of fast beats, slow beats, hysteresis transitions, VRP blocking, and pacing deadlines. This ensured that the pacemaker encountered many different variations of timing patterns, increasing the likelihood of revealing any violations of PropLRI, PropHRI, or PropVRP.

The VentricleController reacts deterministically to these random inputs:

- If time_ms – lastBeat ≥ LRI, it must pace (PropLRI)
- If a sensed event occurs, hysteresis must activate, and RI becomes HRI (PropHRI)
- After every event, VRP must elapse before another event is registered (PropVRP)

Since guard conditions in our code mirror the UPPAAL model, the random input stream tests all the model's transitions and timing boundaries. This makes Random Testing an effective way to stress-test the implementation and detect deviations from the behavior specified and verified in the model.

B. Black-Box Testing

We tested the system in a black-box manner, judging correctness from the monitor log's output rather than internal code structure. Each ventricular event produced by the VentricleController is published to the Observer in the format: [EventType:<interval>]. Because the UPPAAL timing properties (PropLRI, PropHRI, PropVRP) depend solely on these externally observable values, we can determine whether the implementation satisfies each property. This makes the monitor terminal output a test oracle: it provides the complete basis for deciding whether each event sequence satisfies or violates the pacemaker's safety requirements.

Tolerance

Because real hardware may introduce small timing delays, our monitor applies an 8ms tolerance when evaluating PropLRI, PropHRI, and PropVRP. UPPAAL assumes perfect clocks, but Arduino execution and MQTT introduce deviations of a few milliseconds. The 8ms tolerance ensures that the test oracle only flags true violations of the timing properties, while still enforcing the safety thresholds defined in the formal model.

C. Implicit Boundary Testing

Our random testing also implicitly exercised boundary value conditions. Because the RandomHeart generates intervals uniformly between 200 ms and 3000 ms, the system encounters events near the lower boundary (~150 ms), which test PropVRP, and near the upper boundary (~3000 ms), which tests PropLRI. These boundary cases provide additional evidence that the pacemaker behaves correctly under boundary conditions.

VII. Software / Hardware

Hardware

We implemented the RandomHeart and Ventricle controller on two Arduino Nano 33 IoT boards placed on separate breadboards connected through digital I/O wires. The pacerPin on the RandomHeart receives pacing pulses from the VentricleController, while the beatPin sends intrinsic ventricular events back to the controller. The RandomHeart's ledPin blinks with every heartbeat, allowing us to observe system timing behavior in real time.

Arduino Software

- Arduino IDE / PlatformIO for compiling and flashing code.
- FreeRTOS for task scheduling on the VentricleController.
- WiFiNINA for secure WiFi connectivity on both Nanos.
- ArduinoMqttClient for MQTT communication with the Observer.

MQTT Communication

- Topic name: "cis441-541/PacingAhead/Update" → publishes ventricular events to the Observer.
- All MQTT messages follow a "VEventType: interval" format

Observer Software Environment

- C++ program running in Codio environment
- Eclipse Paho MQTT C++ Client for asynchronous event handling
- std::thread for parallel execution

VIII. Assurance Cases + Confidence Argument

Assurance Cases

Our assurance case graphs (Figures 6-9) follow the structure presented in the Assurance Case Example slides. Our top-level goal (G1) is, “the implemented pacemaker software is acceptably safe to operate in VVI mode”. Using Goal Structuring Notation (GSN), our diagram lays out a structured argument demonstrating that the implementation satisfies all required safety properties. We support this goal by addressing three subgoals: (1) the correctness of the formal UPPAAL model, (2) the correctness of the implementation with respect to that model, and (3) the correctness of the implementation under realistic timing uncertainty.

G3 Correctness of the formal UPPAAL model:
This branch argues that the formal timed-automata specification of the pacemaker (RandomHeart, VentricleController, Observer) satisfies all required safety properties. We support this using formal model-checking results (PropLRI, PropHRI, PropVRP, PropURI, deadlock-freedom), and a modeling context that captures the timing assumptions of VVI pacing.

G4 Correctness of the code generation/implementation:
This branch argues that the C++/Arduino implementation is a faithful translation of the verified UPPAAL model. We rely on (a) the structural correspondence between EFSM transitions and code, (b) the Thread-Per-EFSM architecture, and (c) manual inspection of guards and state updates.

G5 Correctness under real-time timing tolerance
Because real hardware does not operate with perfect clocks, this branch demonstrates that the implementation remains safe when executed with a ±8 ms timing tolerance. We justify this tolerance based on the accuracy of the Arduino Nano timer, the latency of FreeRTOS scheduling, and the behavior observed in runtime logs.

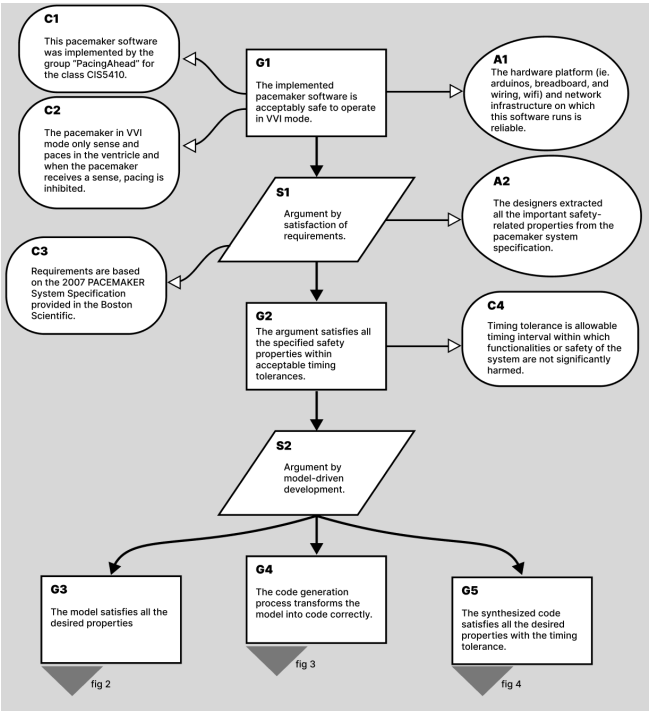

Figure 6: Assurance Case Top-Level Claims

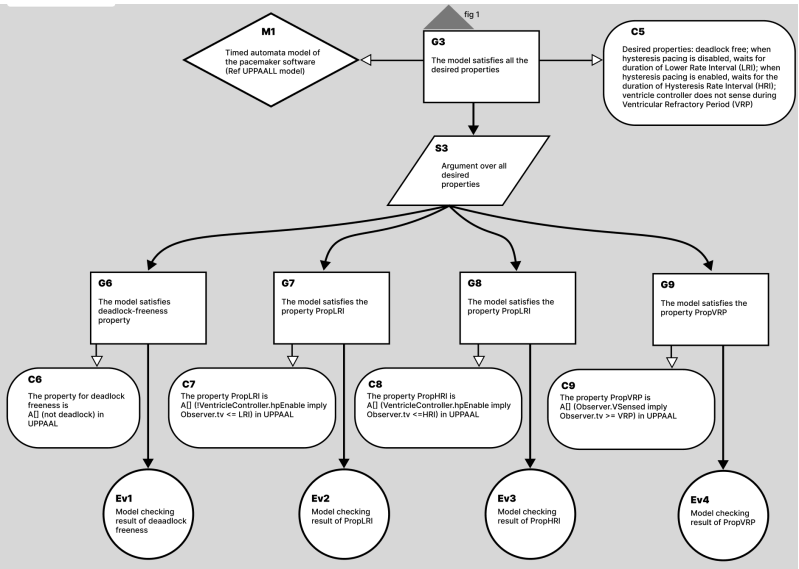

Figure 7: Modeling Claims

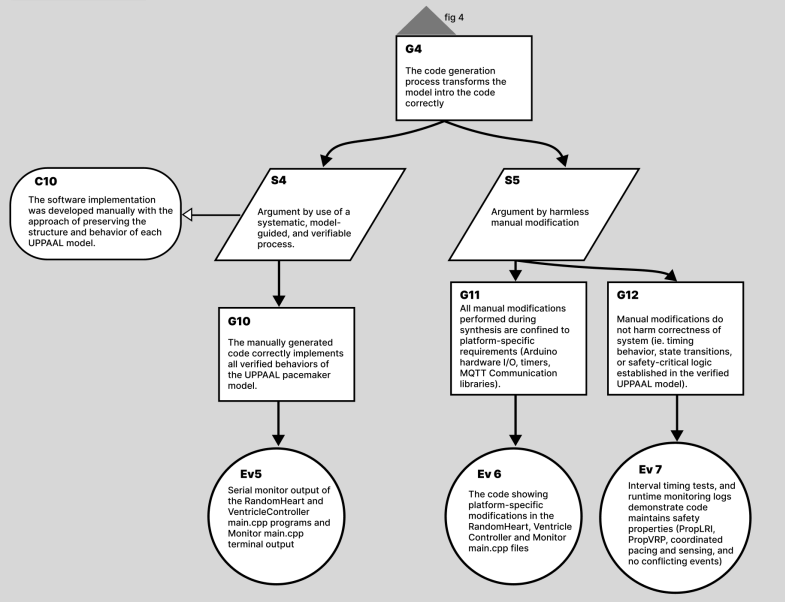

Figure 8: Code Synthesis Claims

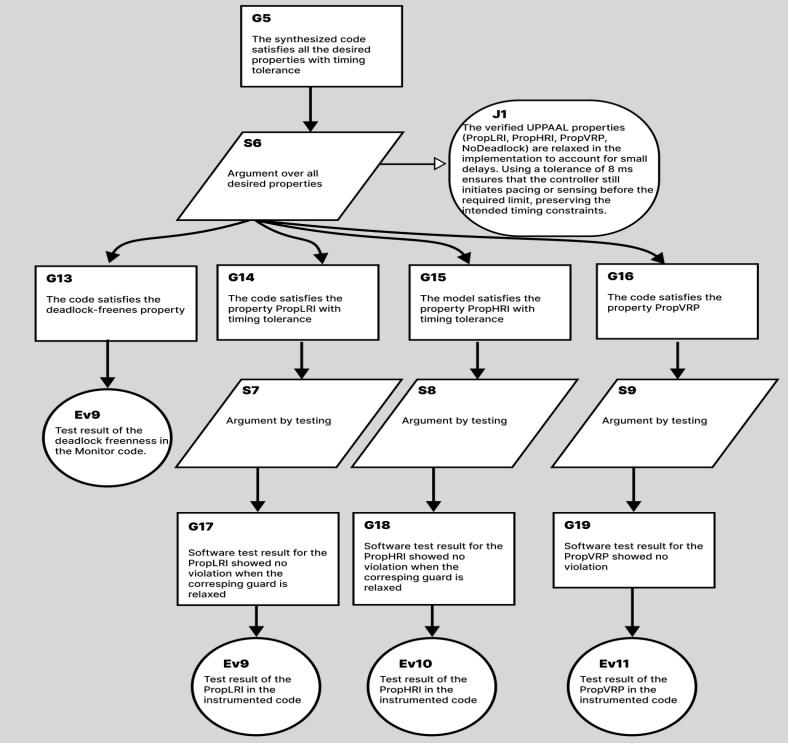

Figure 9: Fault tolerance claims

Confidence Arguments

In our confidence arguments graph (Figure 10), the top-level confidence goal (G1) is: “we have confidence that the safety argument for the pacemaker is correct and reasonably reliable.” This goal is supported by arguments over confidence in evidence, confidence in contexts and assumptions, and confidence in the inferences.

G2 Confidence in Evidence:
This includes confidence that UPPAAL results and runtime logs are trustworthy (Assumption A1), that the evidence is relevant and complete, and that the monitor logs accurately reflect system behavior (Ev1, Ev2).

G3 Confidence in Contexts and Assumptions
This includes assumptions about timer precision (C3), the justification of the ±8 ms tolerance (A2), and validation that all environmental assumptions in the assurance case are realistic for the Arduino hardware.

G4 Confidence in the Inferences
This ensures that the reasoning steps linking the model, code, and test results are logically sound. We support this with justification J1 and evidence Ev3, which confirms that UPPAAL guards and transitions were correctly incorporated into the implementation.

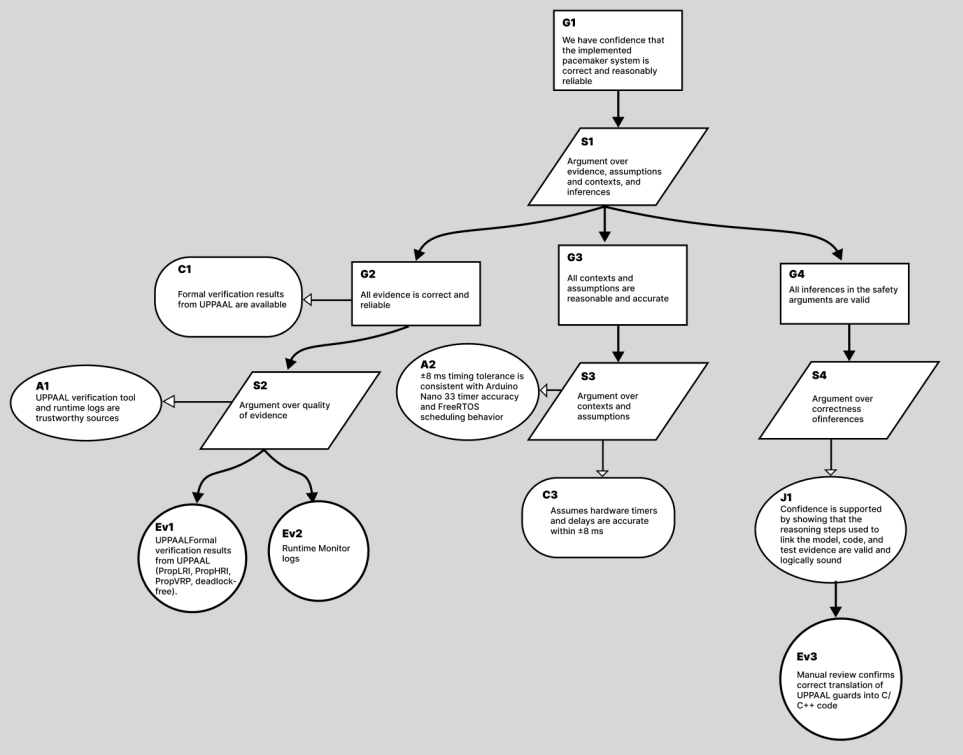

Figure 10: Confidence Arguments

IX. Running the program

1. Hardware Setup
   - Connect the two boards using three digital wires:
   - RandomHeart beatPin (7) → VentricleController sensePin (7)
   - VentricleController pacePin (8) → RandomHeart pacerPin (8)
   - Shared GND between both boards

2. Flashing the Arduino Programs
   - Each Arduino Nano 33 IoT board is flashed separately using the Arduino IDE or PlatformIO. Upload the main.cpp files of RandomHeart and VentricleController to their respective Arduino's
   - After flashing, open the Serial Monitor to confirm that each device initializes correctly and connects (for the controller) to WiFi and begins sensing/pacing in synchronization

3. Starting the Observer (Monitor)
   - In the Codio environment, compile and run the observer using:

```
make main
./main
```

4. Viewing System Behavior

With all components running, you will see:

- The RandomHeart printing natural beats or acknowledgment of paced beats
- The VentricleController prints pacing decisions and sensed natural beats
- The Observer prints event intervals, hysteresis mode status, BPM calculations, and any safety alerts

X. Member Contributions

- Formal Modeling: Sabine
- Code Generation: Krish
- Code Testing: Krish
- Presentation Slides: Sabine
- Write-up: Sabine and Krish